

Final Project Report

ExpenseTracker

A Full-Stack MERN Personal Expense & Budget Management System

Project Title	ExpenseTracker — Personal Expense & Budget Management System
Submitted By	Sharma Saurabh
Roll Number	250863131007
Institute	Laxmi Institute of Technology (LIT), Sarigam — Laxmi Vidyapeeth
Department	Computer Science & Engineering
University	Gujarat Technological University (GTU)
Project Guide	X
Academic Year	2025–2026
Report Date	08 October 2026

Abstract

Managing personal money with notebooks or scattered spreadsheets is slow, error-prone and gives no real picture of spending habits. ExpenseTracker is a web application built on the MERN stack (MongoDB, Express.js, React.js, Node.js) that lets a user record income and expenses, organise them into categories, set monthly budgets, track savings goals, automate recurring payments and understand spending through charts and monthly reports.

The system uses a REST API with JWT authentication, a Redux Toolkit powered React front end, and MongoDB for storage. Beyond basic CRUD, it provides smart de-duplicated notifications, month-over-month reports, CSV export, receipt uploads, tags, favorites, light/dark mode and eight selectable colour themes. All data is strictly per-user and validated on both client and server.

Table of Contents

1. Introduction
2. Objectives
3. Technology Stack
4. System Design
5. Database Design
6. Modules & Features
7. API Reference
8. User Interface
9. Implementation Highlights
10. Security & Validation
11. Testing
12. Challenges & Solutions
13. Limitations
14. Future Scope
15. Conclusion
16. How to Run
17. Appendix — Environment Variables

1. Introduction

1.1 Background

Most people know roughly what they earn but not where it goes. Tracking every expense by hand is tedious, and existing apps are often cluttered, ad-driven or locked behind subscriptions. A simple, private, self-hosted tracker solves this for students and individuals.

1.2 Problem Statement

There is a need for a lightweight web application that lets a user record transactions quickly, stay within monthly budgets, work towards savings goals, and see clear analytics — without relying on third-party services.

1.3 Proposed Solution

ExpenseTracker provides a single dashboard for all of this. Users register, add categories and transactions, set per-category monthly budgets, create savings goals and recurring expenses, and view analytics and monthly reports. The application runs locally with two commands and needs no external APIs.

1.4 Scope

The project covers authentication, transaction management, categories, budgets, savings goals, recurring expenses, analytics, reports, notifications, CSV export, receipt upload and appearance customisation. Bank integration and mobile apps are outside the current scope.

2. Objectives

1. Build a full-stack application covering the complete finance workflow: transactions → budgets → goals → reports.
2. Provide secure user accounts where each user can only access their own data.
3. Give meaningful insight through interactive charts and month-over-month comparison.
4. Warn users automatically when budgets are nearly or fully used.
5. Offer a clean, consistent and accessible interface with light/dark mode and selectable themes.
6. Keep the code simple and modular: small single-purpose files and sequential `async/await` flow.

3. Technology Stack

Layer	Technology	Purpose
Database	MongoDB (local)	Flexible document storage for finance records
Backend	Node.js, Express.js	REST API
ODM	Mongoose 8	Schemas, validation, references
Frontend	React 18 (Vite)	Component-based SPA, fast dev server
State	Redux Toolkit	Predictable state via slices
Routing	React Router v6	Protected routes
Charts	Recharts	Bar, line, area, pie and donut charts
Styling	Tailwind CSS + CSS variables	Consistent styling, theming, dark mode
HTTP	Axios	Central instance with auth/error interceptors
Auth	JWT, bcryptjs	Stateless auth, hashed passwords
Uploads	Multer	Receipt images (max 2 MB)
Security	helmet, CORS allow-list	Secure headers, restricted origins
Feedback	react-hot-toast	Success/error toasts

Backend runs on port `5000` and frontend on `5173` ; the Vite dev server proxies `/api` and `/uploads` to the backend.

4. System Design

4.1 Architecture

ExpenseTracker follows a three-tier client–server architecture: a React single-page application, an Express REST API, and a MongoDB database.

Data flow: Page → Redux slice → `services/api.js` (Axios + JWT header) → Express route → controller → Mongoose model → MongoDB → JSON response → slice state → UI and charts.

4.2 Project Structure

```
ExpenseTracker/
├─ server/
│  └─ src/
│     ├─ config/env.js           # only file reading process.env
│     ├─ models/                 # 7 Mongoose models
│     ├─ middleware/             # auth (JWT), error, upload (Multer)
│     ├─ utils/                  # notify, processRecurring, buildCsv, sanitize
│     ├─ controllers/            # 8 controllers
│     ├─ routes/                 # 8 routers (40 endpoints)
│     ├─ seed/seed.js            # demo data
│     └─ app.js, server.js
└─ client/
   └─ src/
      ├─ config/env.js           # only file reading import.meta.env
      ├─ themes/                 # 8 theme files + registry
      ├─ features/               # 8 Redux slices + store
      ├─ services/api.js         # Axios instance + interceptors
      ├─ layout/                 # Sidebar, Navbar, MainLayout
      ├─ common/                 # Modal, EmptyState, Skeletons, charts
      ├─ pages/                  # 11 pages
      └─ index.css                # CSS variables + component classes
```

5. Database Design

Collection	Key fields
User	name, email (unique), password (bcrypt hash), currency, monthlyIncome, theme preferences

Category	user, name, type (income/expense), icon, color
Transaction	user, type, amount, category (ref), description, date, paymentMethod, tags[], receipt, favorite, notes, isRecurring, recurringExpense
Budget	user, category (ref), amount, month, year
SavingsGoal	user, title, targetAmount, currentAmount, targetDate, icon
RecurringExpense	user, title, amount, category, frequency, nextDate, paymentMethod, active
Notification	user, type, title, message, key (for de-duplication), read

Finance collections are indexed on `user` with date or month/year for fast aggregation. Every query filters by the logged-in user's id, so users can never see each other's data.

6. Modules & Features

#	Module	Description
1	Authentication	Register, login, fetch current user, update profile (name, currency, monthly income target)
2	Dashboard	Total balance, monthly income/expense, category donut, budget status bars, recent transactions
3	Transactions	Add/edit/delete income and expenses; search, pagination; filters by date range, category, type, payment method, amount range and favorites; tags
4	Categories	Custom categories with icon and colour; separate income and expense types; deletion blocked when used by a recurring expense
5	Budgets	Per-category monthly limits with progress bars and on-track / near-limit / over-budget status
6	Savings Goals	Target vs current amount, add-money contributions, progress percentage
7	Recurring Expenses	Monthly payments with active toggle; due items are posted automatically as real transactions
8	Analytics	Income vs expense by month, spending by category, payment-method split; switch between Bar, Line, Area, Donut and Pie charts
9	Monthly Reports	Month navigator, month-over-month insight banner, six-month chart, category table with change %, top categories
10	Notifications	Budget warning ($\geq 80\%$), budget exceeded ($\geq 100\%$), goal milestones, recurring-due and welcome alerts, de-duplicated by a unique key
11	CSV Export	Downloads filtered transactions as a date-stamped CSV
12	Receipts & Favorites	Attach an image (≤ 2 MB) to a transaction with preview; star frequent transactions
13	Appearance	Light/dark mode and eight themes, saved in the browser

7. API Reference (40 endpoints)

Router	Endpoints
/api/auth	POST /register, POST /login, GET /me, PUT /profile
/api/transactions	GET /, POST /, GET /:id, PUT /:id, DELETE /:id, GET /csv, GET /favorites, PATCH /:id/favorite, PATCH /:id/receipt
/api/categories	GET /, POST /, PUT /:id, DELETE /:id
/api/budgets	GET /, POST /, PUT /:id, DELETE /:id
/api/goals	GET /, POST /, PUT /:id, DELETE /:id, POST /:id/add-money
/api/recurring-expenses	GET /, POST /, PUT /:id, DELETE /:id, PATCH /:id/toggle
/api/analytics	GET /dashboard, /monthly, /categories, /payment-methods, /reports
/api/notifications	GET /, PATCH /read-all, PATCH /:id/read, DELETE /:id

8. User Interface

8.1 Design System

The default theme uses a warm white background with espresso brown and sage green accents. All colours are CSS variables (`--bg` , `--text` , `--card` , `--primary` , `--accent` , ...) defined for both light and dark mode, so changing theme or mode only swaps variables and no component needs colour logic. The full interface uses the JetBrains Mono typeface.

Token	Hex	Usage
Background	#FAF8F3	Page background
Text	#29231F	Primary text
Primary (Espresso)	#493B32	Buttons, active states
Accent (Sage)	#78816A	Charts, highlights
Sage tint	#E9EDE4	Sidebar, soft panels
Beige tint	#E9DFD1	Balance and stat cards

8.2 Theme Engine

Each of the eight themes lives in its own file under `client/src/themes/` , with a small registry exposing `getTheme()` and `applyTheme()` . The choice is made under Profile → Appearance and persisted in `localStorage` . Dark mode is an independent toggle available in the navbar.

8.3 Interaction & Accessibility

Transitions (150–250 ms) are pure CSS: hover lift, modal fade, progress-bar fill, skeleton shimmer and a flash on newly added rows. Modals close with Escape, inputs are labelled, and focus outlines are visible. Amount inputs accept only digits, one dot and two decimals. Every page has loading skeletons and a friendly empty state.

8.4 Screenshots

Insert your own screenshots in the boxes below (replace each box with an tag).

Screenshot: Login page

Figure 1 — Login

Screenshot: Dashboard

Figure 2 — Dashboard with balance, charts and budget status

Screenshot: Transactions list and Add Transaction modal

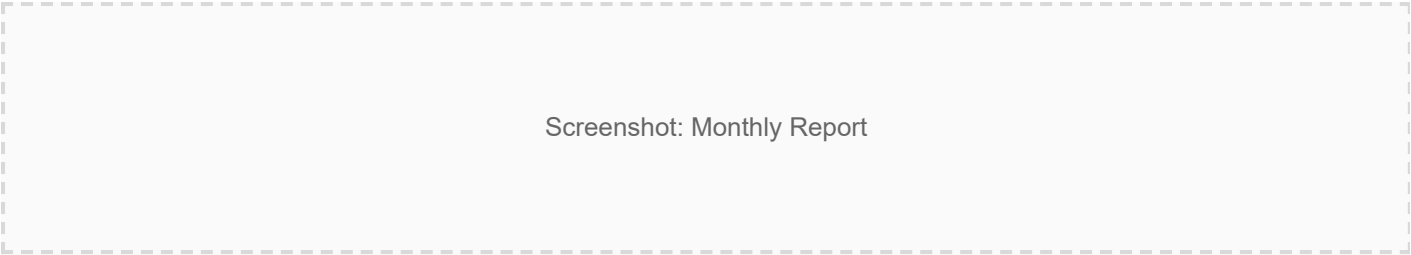
Figure 3 — Transactions

Screenshot: Budgets

Figure 4 — Monthly budgets

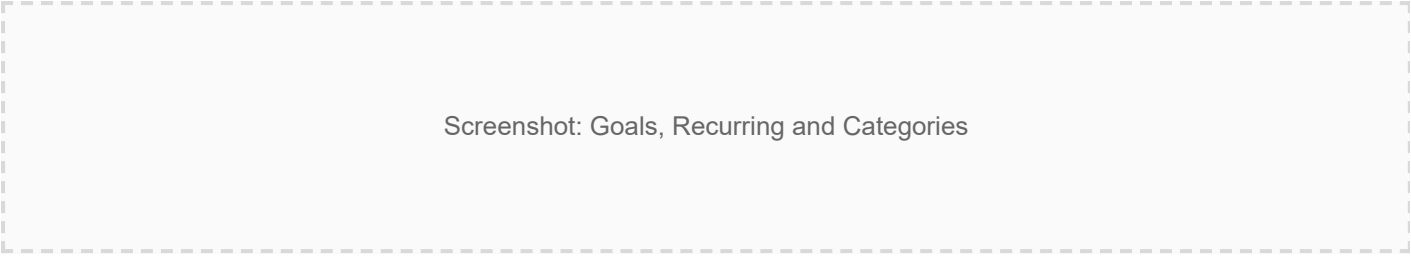
Screenshot: Analytics (Bar and Line views)

Figure 5 — Analytics with chart-type selector



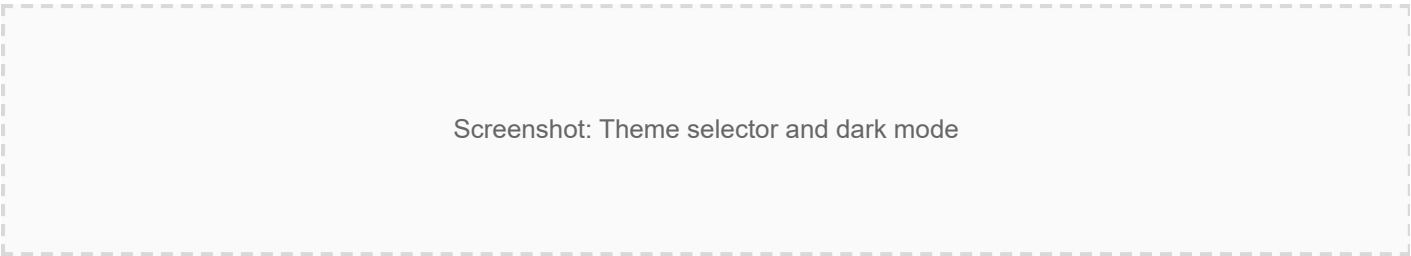
Screenshot: Monthly Report

Figure 6 — Monthly report with month-over-month insight



Screenshot: Goals, Recurring and Categories

Figure 7 — Savings goals, recurring expenses and categories



Screenshot: Theme selector and dark mode

Figure 8 — Appearance settings and dark mode

9. Implementation Highlights

9.1 Recurring Auto-Posting

When the app loads, every active recurring item whose `nextDate` is today or earlier is converted into a real transaction, its next due date is advanced, and a notification is created.

```
const dueItems = await RecurringExpense.find({
  user: userId, active: true, nextDate: { $lte: today },
});
for (const item of dueItems) {
  await Transaction.create({ user: userId, type: 'expense',
    amount: item.amount, category: item.category,
    description: item.title, date: item.nextDate,
    paymentMethod: item.paymentMethod, isRecurring: true,
    recurringExpense: item._id });
  item.nextDate = nextDateAfter(item.nextDate, item.frequency);
  await item.save();
}
```

9.2 Smart Notifications with De-duplication

Each alert carries a stable key such as `budget:2026:10:Food:warn`. Before creating a notification, the server checks whether that key already exists, so the same warning never repeats.

```
const createOnce = async (userId, type, title, message, key) => {
  const alreadySent = await Notification.findOne({ user: userId, key });
  if (alreadySent) return;
  await Notification.create({ user: userId, type, title, message, key });
};
```

9.2.1 Dashboard and Report Aggregation

Totals, monthly series and category breakdowns are computed with MongoDB aggregation pipelines (`$match` by user and date, then `$group`), executed with sequential `await` calls for readability.

9.3 Amount Validation

The client strips invalid characters while typing and limits to two decimals; the server independently converts and validates the amount and returns HTTP 400 for values such as `abc` , `-5` or `0` .

9.4 Chart Selector

The analytics data stays loaded while only the chart component changes, so switching between Bar, Line, Area, Donut and Pie is instant.

9.5 CSV Export

`GET /api/transactions/csv` applies the same filters as the list and returns a `text/csv` response with a date-stamped filename.

10. Security & Validation

1. Passwords are hashed with bcrypt (10 rounds) and never returned by the API.
2. JWT with configurable expiry is sent as an `Authorization: Bearer` header; the Axios interceptor attaches it and clears it on failure.
3. Ownership checks: every find, update and delete includes the logged-in user's id.
4. Server-side validation through a `sanitize.js` helper returns 400 with clear messages for invalid amounts, empty titles and bad enum values.
5. helmet security headers and a CORS allow-list.
6. Multer restricts uploads to image MIME types up to 2 MB.
7. All environment variables are read in exactly two files; secrets stay in a git-ignored `.env`.

11. Testing

Area	Test	Result
Auth	Register, login, get current user	Pass (200)
Dashboard	Summary aggregation	Pass (200)
Reports	Month-over-month, categories, payment methods	Pass (200)
CRUD	Transactions, budgets, goals, recurring, categories	Pass (200)
CSV	Export endpoint	Valid CSV stream
Validation	Amount <code>abc</code> , <code>-5</code> ; budget <code>0</code>	Rejected with 400
Validation	Amount <code>123.456</code>	Stored as 123.46
Categories	Delete category used by a recurring expense	Blocked
Frontend	<code>npm run build</code>	Success (948 modules)
Seeding	Demo and Saurabh users with data	Verified

12. Challenges & Solutions

Challenge	Solution
MongoDB error “field must be an accumulator object” in <code>\$group</code>	Used accumulators such as <code>{ \$first: '\$name' }</code> for non- <code>_id</code> fields in three pipelines
Custom <code>.table-row</code> class clashed with Tailwind's <code>display: table-row</code>	Renamed the component class to <code>.row-hover</code>
Amount fields accepted letters, negatives and extra decimals	Added <code>sanitizeAmountInput</code> on the client and <code>toAmount</code> on the server
Port conflicts (<code>EADDRINUSE</code>) on restart	One dev process per port, ports defined in <code>.env</code>
Environment variables scattered across files	Restricted raw access to two config files

13. Limitations

- Recurring expenses are posted only when the app is opened, not by a background scheduler.

- The production build shows a chunk-size warning (>500 kB) because Recharts bundles all chart types.
- Only monthly frequency and a single currency per user are supported.
- No automated test suite yet; testing was done manually and with API calls.

14. Future Scope

1. Cron scheduler (node-cron) to post recurring expenses without opening the app.
2. Monthly report emails and budget breach push/email alerts.
3. Bank statement (CSV/OFX) import and receipt OCR.
4. Smart features: automatic category suggestion, anomaly detection and month-end forecasting.
5. Multi-currency support and shared/family budgets.
6. PWA with offline mode and a React Native mobile app using the same API.
7. Rate limiting, refresh-token rotation and an audit log for transactions.
8. Jest + Supertest API tests, CI/CD, and route-level code splitting.

15. Conclusion

ExpenseTracker delivers an end-to-end MERN application that covers the full personal-finance workflow — transactions, budgets, savings goals, recurring payments, analytics and reports — with secure per-user data, validation on both client and server, and a themeable, accessible interface. The modular code structure keeps the project easy to understand, maintain and extend, and the future-scope items provide a clear path for growth.

16. How to Run

Prerequisites: Node.js 18 or later and MongoDB running locally on port 27017.

```
# Backend
cd server
npm install
cp .env.example .env
npm run seed      # creates demo data
npm run dev       # http://localhost:5000

# Frontend (new terminal)
cd client
npm install
npm run dev       # http://localhost:5173
```

Demo logins: `demo@example.com` / `password123` (31 transactions, 5 budgets, 5 recurring expenses, 3 goals)
and `saurabh@gmail.com` / `saurabh123` (18 transactions, 3 budgets, 3 recurring expenses, 3 goals).

Appendix — Environment Variables

server/.env

```
PORT=5000
MONGODB_URI=mongodb://127.0.0.1:27017/expenseTracker
JWT_SECRET=<your-secret>
JWT_EXPIRES_IN=7d
CLIENT_URL=http://localhost:5173
```

client/.env

```
VITE_API_BASE_URL=/api
VITE_APP_NAME=ExpenseTracker
VITE_APP_TAGLINE=Track every rupee
VITE_REQUEST_TIMEOUT=15000
```